Communic-Aid

Adapting Dwell Time and Confirmation in Gaze-Based Messaging with a Commodity iPhone

CS 435: Human-Computer Interaction
Assignment 3: High-Fidelity Prototyping and User Evaluation
Group 4: Vandya Bisaria (25510145), Mansie Singh (25510068), Sinngam Khaidem (25250040)
Instructor: Dr. Yogesh K. Meena, IIT Gandhinagar
12 November 2026

Introduction

Communic-aid is a working, software-based high-fidelity prototype of a hands-free messaging app for people with motor impairments. An iPhone with a TrueDepth (Face ID) camera tracks gaze, head pose, blinks and eyebrow raises, and listens for the word “go”; a switch button can also be used. A Python core on a computer turns these into selections in a browser-based chat app with messages, calls and a gaze keyboard. The core idea is that the interface learns how much evidence to ask for before it acts, per person, per context and per level of risk: a dwell learner sets how long a look must last, and a modality router decides which confirmation signal to ask for on risky actions. We evaluated the prototype with a heuristic evaluation and a controlled usability test with three participants, one of them from the target group. Code, README, demo video and evaluation data: https://github.com/sinngam-khaidem/communic-aid.

1 System Design and Architecture

1.1 Problem and goal

For many people with ALS, muscular dystrophy, cerebral palsy or spinal cord injury, the eyes remain controllable long after the hands. Gaze interfaces suffer from the Midas touch problem [4]: the eyes are always looking at something, so the interface cannot tell “do this” from “I am reading”. The usual remedy is a *dwell* time: a button is selected only after the eyes have stayed on it long enough. A long dwell is safe but slow; a short dwell is fast but selects things by mistake. Adaptive dwell methods tune one threshold for all actions [6, 8], and multimodal methods pair gaze with a fixed second input [9]. Our earlier reports identified three gaps: confirmation does not scale with the cost of an error, dwell adaptation rarely learns from implicit signals, and no system chooses the confirmation modality from experience. Communic-aid addresses all three with one question: *how much evidence should the interface ask for before it acts?*

1.2 Architecture

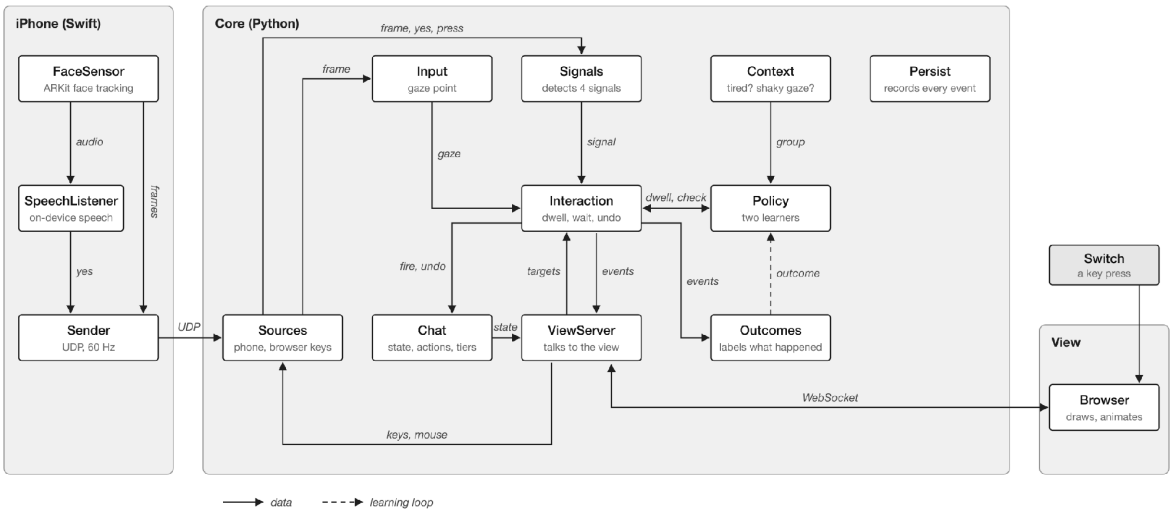


Figure 1. Architecture. The iPhone only senses, the Python core makes every decision, and the browser only draws. The dashed arrow is the learning loop: outcomes update the policy.

The system has three parts (Figure 1):

1. **iPhone app** (Swift, ARKit face tracking). Sends, 60 times a second over UDP, the gaze ray and eye position in camera space, the nose direction, and blink and eyebrow blendshapes. On-device speech recognition listens only for the word "go". A live face preview helps the person position themselves.
2. **Core** (Python, asyncio). Maps gaze to the screen with a nine point calibration (quadratic least squares with a small ridge term), smooths it with an exponential filter, holds it still during blinks, and corrects slow drift from the centres of buttons just selected. It detects signals with per-person thresholds set halfway between rest and peak in a signal check (blink held 500 ms, eyebrows raised 300 ms, the spoken word, or a switch key). It runs the dwell and confirmation logic, labels outcomes, updates the learners and logs every event.
3. **View** (HTML, CSS and JavaScript over a WebSocket). Draws the screens and animations and holds no logic.

Why an iPhone and not a webcam. The TrueDepth camera projects infrared dots and builds a depth map of the face, so ARKit gives eye position and head pose in millimetres and degrees, in any lighting; an ordinary laptop webcam gives only a flat colour image. The phone only senses: the targets live on the larger laptop screen, where buttons can be big and well spaced. There is no external API and all logic runs locally, so face and gaze data never leave the two devices. Table 1 lists the technologies.

Table 1. Technologies used.

Part	Technology
Sensor app	Swift, SwiftUI, ARKit face tracking, Apple Speech framework, UDP
Core	Python 3.11, asyncio, aiohttp.
View	HTML, CSS, JavaScript in a full screen browser window
Analysis	Python scripts with Matplotlib: heatmaps, dwell plots, summaries
Testing	pytest, 79 automated tests; a mouse mode that runs without the phone

1.3 Risk tiers

Every action belongs to a risk tier whose accident cost doubles with each step (Table 2). Both learners use this cost.

Table 2. Risk tiers. Tiers 2 and 3 add a confirmation step after a fixed 1000 ms dwell.

Tier	Actions	Accident cost	Confirmation
0	Open a chat, back, scroll, navigation	1	Learned dwell
1	Letters, word suggestions, Space, Backspace, settings	2	Learned dwell
2	Send, react, end a call	4	Router: dwell alone or a signal
3	Call, delete, forward	8	Always a signal chosen by the router

1.4 Implicit outcomes

The learners never ask the person for feedback. The core labels each look and each confirmation request from what the person does next (Table 3). Statistics are kept per context group: rested or tired (over 20 minutes in, or after 6 pm) crossed with steady or shaky gaze (recent gaze spread over 1.5 times a steady baseline).

Table 3. Implicit outcomes.

Outcome	Rule	What it tells the learner
Accident	A selection undone within 5 s, or a letter deleted within 3 s	The dwell was too short
Worked	A selection that was not undone	The dwell was long enough
Give-up	A look filled at least half the dwell, then the same button was chosen afresh within 5 s	The dwell was too long
Stray glance	Any other look that did not fill	Normal reading; how long glances last
Blocked	A confirmation request timed out with no attempt	The check stopped an unwanted action

1.5 Learning

Dwell learner (tiers 0 and 1). From counts of intended and unintended looks, stray glances and give-ups, it draws by Thompson sampling [12] the probability π that a look is intended, a mean stray glance length g and a mean patience p, and scores each dwell d from 800 to 2000 ms:

$$V(d) = \pi\left[(1 - G(d))(1 - \lambda d) + r_{give-up}\, G(d)\right] - (1 - \pi)\, A(d)\, c_{tier}$$

$$A(d) = e^{-d/g}, \qquad G(d) = 1 - e^{-d/p}$$

where:

- $A(d)$ is the chance a stray glance lasts long enough to fire, and $G(d)$ the chance the person gives up before the button fills;
- $\lambda = 0.6$ per second is the cost of waiting, $r_{give-up} = -0.5$ the reward when the person gives up, and c_{tier} the accident cost of the tier.

In words: a longer dwell stops more stray glances from firing, but makes the person wait and sometimes give up; the learner picks the dwell with the best balance, and a higher tier cost pushes it towards caution. After each selection the dwell moves 5% towards the best d, by at most 10 ms when shortening and up to 100 ms when lengthening, so it shortens slowly but recovers quickly after an accident. It starts at 1500 ms and holds for 20 selections per group.

Modality router (tiers 2 and 3). A contextual bandit [5] keeps, per signal, tier and group, counts of worked, give-up, blocked and accident outcomes, samples their probabilities from a Dirichlet distribution and asks for the signal with the highest expected reward. A worked signal earns 1 minus its effort, which grows with the person's time to produce it; a blocked accident earns a small reward; an accident costs the tier cost. The person's preferred signal starts with a bonus.

Constants. The tier costs, λ, $r_{give-up}$, the dwell range, the warm-up and the step sizes were set by hand during design and pilot sessions and kept the same for every participant; none was tuned per person. The doubling cost is an ordinal choice: each tier is taken to be about twice as bad to trigger by mistake as the one below.

2 Compromises in Prototyping

We built the prototype wide enough to show every screen of a messaging app (horizontal), and deep along the one path that carries our research question: from the camera to the learners (vertical). Table 4 compares the two.

Table 4. Horizontal and vertical scope of the prototype.

	Horizontal prototype (breadth)	Vertical prototype (depth)
Scope	Every screen: onboarding, signal check, calibration, chat list, thread, compose, call, message actions, settings, typing test	The full path from sensing to learning, working end to end with real users
What is real	Navigation, layout, feedback, undo, tier colours, both keyboards, word suggestions and quick replies	Gaze and head pointing, signal detection, dwell and confirmation, outcome labelling, both learners, per-person saving, logging and analysis
What is simulated	Contacts, chats and replies come from a local file; replies arrive after 7 s; calls connect after 3 s with no audio	Nothing on this path is simulated
What it tests	Whether the layout, navigation and feedback hold together	Whether risk tiers and adaptive dwell work for a real task

Other compromises we made:

- The phone is a sensor for a computer screen rather than a standalone app. This keeps targets large and lets us log everything, but adds a second device.
- An iPhone with Face ID is required, and apps signed with a free developer account expire every 7 days.
- Speech recognises one word, "go", rather than dictation, which our interviewee with slurred speech found unreliable.
- Word suggestions come from a fixed word list, not a language model, so no AI API is needed.
- Head pointing is built but not yet evaluated, and the tired flag uses a simple rule rather than a fatigue measure.

3 Conceptual Design

3.1 Metaphor

Communic-aid borrows the **messaging app** people already know: chats, threads, a call button, a draft box and Send, so no new model of communication must be learned. Selection follows a **press and hold** metaphor: looking at a button presses it, and the button fills like a progress bar until it fires, so a look can always be abandoned before it counts. Risky buttons add a **safety catch**, like the second step of a slide to unlock: after the fill, the button shows the signal it needs (an eye, an eyebrow, "go"), and nothing happens without it.

3.2 Conceptual model

The objects are chats, messages, contacts and a draft; the actions are open, type, send, react, call, delete, forward and undo. Each action carries a tier, and the tier, not the screen, decides how much evidence it needs.

The user controls what to do, which signals are on and which comes first; the system controls how long to wait and when to ask for a signal, and adjusts both from how the person uses it. Every action can be undone for 5 seconds, and losing the face never fires anything.

The framing behind this is the extended mind thesis [3]: a tool that is reliably coupled to a person's perception and action can become part of their cognitive system [2]. Communic-aid aims for that coupling. The person's own way of attending, their gaze, drives the pointer directly, so looking and acting form one perception and action loop. The screen it drives can be any display, from a laptop to a classroom smartboard.

3.3 Design principles

From our interviews with three people with motor impairments and the literature:

1. Calm and low effort: nothing rushes or startles the person.
2. Every visual element gives feedback or prevents a mistake.
3. The cost of an error decides the check.
4. Learn from implicit behaviour, never from ratings.
5. Reading is safe: text is never a target, only buttons are.

3.4 Design space

Table 5 lists the dimensions we explored and our choices. The choices favour mainstream hardware [14, 7] and per-person adaptation, in the spirit of ability-based design [13].

Table 5. Design space.

Dimension	Options considered	Chosen
Sensor	Eye tracker, webcam, iPhone TrueDepth	iPhone TrueDepth: cheap, mainstream, gives depth and head pose, unlike a webcam
Pointing	Gaze, head, gaze and head	Gaze, with head pointing as an option
Selection	Dwell, blink, smooth pursuit, gaze gestures	Dwell with a learned time
Confirmation	None, fixed second signal, user chosen, learned	Learned per tier and context
Signals	Blink, eyebrows, voice, switch, EMG, BCI	Blink and hold, eyebrows, "go", switch
Adaptation	Explicit ratings, implicit outcomes	Implicit outcomes
Text entry	QWERTY, grouped letters, dwell-free, AI text	Grouped (default) and QWERTY, with suggestions and quick replies
Feedback	Visual, sound, colour coding	All three

4 Prototype Description and Functionalities



Figure 2. Screens of the working prototype, cropped to the active area. The small pink dot in (f) is the gaze pointer. In (e) the outlines show the tiers: amber for React (tier 2), red for Delete (tier 3). In (f) Delete has filled and now shows a closed eye: it waits for blink and hold before it deletes.

Figure 2 shows the main screens. The core interaction flow is:

1. Look at a button. It fills as long as the look stays on it.
2. When it fills, a tier 0 or 1 action happens at once.
3. On a tier 2 or 3 button, the button shows the signal it needs, and the person gives it (or looks away to cancel).
4. After any action, an undo strip appears for 5 seconds.

Navigation. Chats and Settings sit in a fixed side bar; Back, scroll buttons and the undo strip are in the same places on every screen. A chat opens a thread; More on a message opens its actions (React, Forward, Delete); Call opens a call screen with End call.

Text entry. The grouped keyboard has eight letter groups (abc to wxyz): a group, then a letter (Figure 2c, d). QWERTY is one selection per letter. Both offer word suggestions and five quick replies.

Speech input. The system listens for one word, "go", which works as a confirmation signal. When a tier 2 or 3 button fills, saying "go" fires the action. The router treats it like blink and hold or eyebrow raise, and learns how often it works for each person.

Button input. A switch key is the fourth confirmation signal. Gaze points, and a press confirms, following the gaze-plus-switch. On tier 2 and 3 actions, the router can ask for the switch like any other signal. Three presses always escape to a safe state, even if face tracking fails.

Feedback and transitions. A pink dot shows where the system thinks the person is looking. Buttons fill as they are looked at and glow when they fire; tier 2 buttons have an amber outline and tier 3 a red one. A confirmation request shows the signal's icon on the button and waits up to 4 s; a wrong signal makes the button shake and plays a soft error tone, and a confirmed one plays a tick. Plain language banners appear when tracking fails ("I can't see your eyes. Look at the screen.", "Move a little closer"), only after the problem has lasted a second, so they never flicker.

Setup and settings. First use asks three onboarding questions, then runs the signal check and calibration (redone automatically if the error exceeds 100 px). Settings turn signals on or off, choose the one to try first, switch sounds and keyboard, and rerun calibration or the signal check. Three switch presses always escape to safety.

Study tools. A typing test mode shows a passage and measures words per minute for two minutes from the first letter. A terminal log shows every learner decision live. Scripts produce per-person heatmaps, dwell plots and summary tables.

The Extended Mind Hypothesis

The prototype is based on the Extended Mind hypothesis (Clark and Chalmers, 1998). It couples gaze and cursor movement and utilises it as a perception-action loop between the biological brain and external tools (Chalmers, 2019). This allows the person to use their eyes to control external objects and perform various tasks. In the current project, an Apple Macbook Pro is used to test and showcase the high fidelity prototype. However, the extended display can be any type of display - from laptops to smartboards etc. and can be used in a variety of contexts to help individuals with motor impairments. For example, it can be used by educators with motor impairments to perform various tasks on smartboard while teaching.

The rationale behind the use of gaze tracking to control communication apps is therefore based on extended cognition which augments a person's cognitive abilities by utilising different tools.

5 User Evaluation and Results

5.1 Evaluation methods and data gathering

We used two methods. First, a **heuristic evaluation** against Nielsen's ten heuristics [10], done by the team before testing (Table 6). Second, a **task-based usability test** in a **controlled setting**: a quiet room, the laptop on a desk, the phone on a tripod behind the laptop, its front camera facing the participant over the top of the screen at 40 to 60 cm, the participant's head free to move, and an experimenter present. Data for the usability test came from four sources: the system's event log (every look, selection, outcome and learner update); a two minute typing test, run three times with each keyboard; the System Usability Scale (SUS) [1], filled in after the session; and open feedback from each participant.

Table 6. Heuristic evaluation (Nielsen [10]). The team evaluated its own system, a known weakness of the method: the evaluators know how it is meant to work and may be lenient.

Heuristic	Status	Evidence	Planned fix
1. Visibility of system status	Met	Gaze dot, fill on every button, tier outlines, banners when tracking fails	
2. Match with the real world	Met	Familiar messaging words and layout: chats, Send, Call	
3. User control and freedom	Partial	Undo for 5 s, Back, three presses escape; but no way to pause or recentre a drifting pointer (P3 asked for this)	A "reset pointer" action
4. Consistency and standards	Met	Same fill, same positions for Back and undo, same tier colours on every screen	
5. Error prevention	Partial	Risky actions need a second signal and no tier 2 or 3 accident occurred; but QWERTY keys are about the size of the calibration error and corner targets were hard to hit	Larger keys; keep targets off corners
6. Recognition rather than recall	Met	Labelled buttons; the needed signal's icon appears on the button	
7. Flexibility and efficiency	Partial	Dwell and signal adapt to the person; but pointer smoothing is fixed and typing is slow (0.6 to 2.2 wpm)	Smoothing slider; better word prediction
8. Aesthetic and minimalist design	Partial	Few, large buttons and no clutter; but the dark, low contrast theme was criticised (P3)	A high contrast theme
9. Recover from errors	Met	Plain language banners, shake and tone on a wrong signal, undo on every action	
10. Help and documentation	Partial	Onboarding and a hint after 30 s of being stuck, but no help screen	A help screen on signals and recovery

5.2 Participants

Three adults have completed the test, all using gaze pointing with the adaptive learners: two able-bodied participants (P1, P2) and one participant from the target group, with a motor impairment (P3). Two more able-bodied participants are scheduled.

5.3 Tasks

After the signal check and calibration, each participant used the app freely and then completed the tasks in Table 7. Free use lasted 9 minutes for P1, 24 for P2 and 19 for P3.

Table 7. User tasks.

#	Task	Done when
1	Open the chat with Priya, go to Settings and change the preferred signal, go back to the chats	Chat list showing, new signal saved
2	In Priya's chat, type "see you at five" and send it	The message is sent
3	Call Priya; when the call connects, end it	Back in Priya's chat
4	Delete the message "Yes, all good here."	The message is gone
5	Typing test: copy a passage for two minutes, three times with each keyboard	Words per minute shown

5.4 Results

Table 8. Free use session, from the event log. Selections exclude undo and redo.

Measure	P1	P2	P3
Calibration error	80 px	69 px	32 px
Selections (tier 0 / 1 / 2 / 3)	51 (18 / 29 / 2 / 2)	95 (32 / 51 / 7 / 5)	82 (27 / 46 / 6 / 3)
Accidents	2 (tier 1, undone)	0	0
Dwell give-ups	6	11	19
Stray glances	145	541	428
Confirmation requests (worked / gave up)	5 (4 / 1)	13 (12 / 1)	10 (9 / 1)
Dwell at end, tier 0 / tier 1	1216 / 1333 ms	1219 / 1431 ms	1285 / 1574 ms

Accidents and confirmation. Across 228 selections there were two accidents, both letters, both undone (Table 8). No tier 2 or 3 action fired by mistake, and 25 of 28 confirmation requests succeeded. When P1 gave up on raised eyebrows to confirm Send, the next Send asked for blink and hold, which worked. P2 began with blink and hold; after three successful confirmations the router accepted dwell alone on tier 2, and after P2 made "go" the preferred signal, P2 confirmed it five times out of five. P3 needed two tries with "go" on the first call, after

which the router accepted dwell alone on tier 2 (five times, all worked) and asked for blink and hold on tier 3. These counts are small: the router keeps separate statistics per signal, tier and context group, and each participant made only 5 to 13 requests, so most cells hold one to three tries. Its choices so far rest largely on the preferred signal bonus and pooled data; they show plausible behaviour, not learned preferences.

Dwell learning. Stray glances outnumbered selections three to six times: a gaze interface produces much implicit evidence even when nothing is selected. After its warm-up the learner shortened tier 0 from 1500 to about 1220 ms for P1 and P2 and 1285 ms for P3 (Figure 3). Tier 1 stayed longer, as expected from its higher cost; for P3 it rose slightly, to 1574 ms, and P3's shaky gaze group (15 selections) never left the warm-up. Both panels in Figure 3 are "tired" groups because all of P1's sessions were after 6 pm; the plots for P2 and P3 are in the evaluation data.

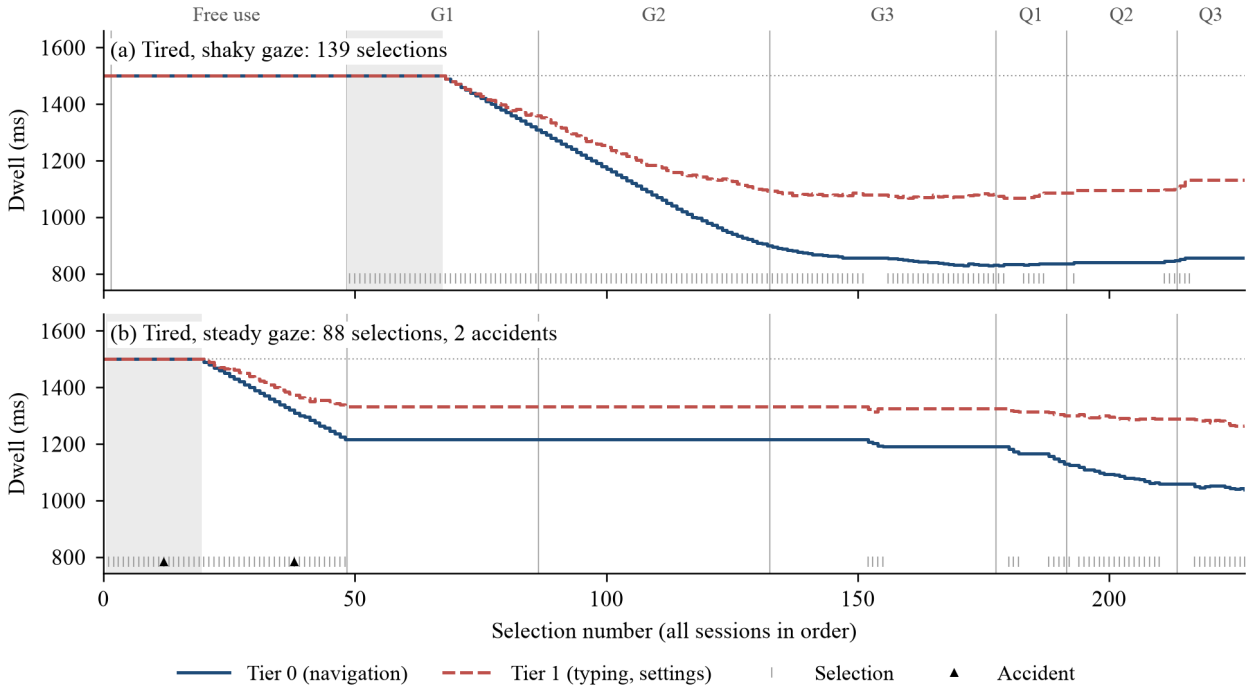


Figure 3. P1's dwell over all sessions, one panel per context group and one line per tier. Sessions run in order: free use, three grouped keyboard typing runs (G1 to G3) and three QWERTY runs (Q1 to Q3); grey vertical lines mark where each session starts. The dotted line is the 1500 ms start, shading marks the warm-up, and ticks along the bottom are this group's selections, with triangles for accidents.

Did adaptation help? Earlier studies show that adapting the dwell time makes gaze typing faster, whether users set it themselves [6] or the system changes it automatically for each key [8]. We have not yet compared our learner with a fixed dwell, so as a first check we split each free use session into two halves. In the second half everyone selected about twice as fast (P1 5.1 to 10.3, P2 3.5 to 6.6, P3 3.3 to 7.5 selections per minute), and accidents did not rise (P1 had one in each half; P2 and P3 had none). The learner shortened the dwell a little for P1 (1500 to 1415 ms) and P2 (1493 to 1435 ms), but not for P3 (1500 to 1522 ms). People also gave up on a look more often in the second half, because they were moving faster than the dwell. The dwell changed too little to explain the speed-up, so most of it came from people getting used to the app; the two halves also held different tasks. What this shows is that shortening the dwell did not lead to more mistakes. Whether our learner makes the app better than a fixed dwell needs the planned comparison.

Typing. The grouped keyboard was 1.5 (P1), 2.8 (P2) and 1.3 (P3) times faster than QWERTY (Table 9), despite almost two selections per letter. Target size explains it: a letter group was 316 × 113 px and a QWERTY key 113 × 67 px, while the calibration error of P1 and P2 was 69 to 80 px, about one QWERTY key. P3, with the smallest calibration error (32 px), was the closest on the two keyboards. We report keystrokes per character as in [11].

Table 9. Typing test, three two minute runs per keyboard, grouped first. P1 and P2 typed with the learning dwell; P3 typed with a fixed 1250 ms dwell.

Participant	Keyboard	Runs (wpm)	Mean wpm	Accuracy	Keystrokes per character
P1	Grouped	2.0, 2.4, 2.3	2.23	100%	1.84

Participant	Keyboard	Runs (wpm)	Mean wpm	Accuracy	Keystrokes per character
P1	QWERTY	1.1, 2.1, 1.3	1.50	91.7%	1.06
P2	Grouped	1.4, 1.6, 1.8	1.60	100%	1.86
P2	QWERTY	0.3, 0.5, 0.9	0.57	100%	1.11
P3	Grouped	1.6, 2.1, 2.1	1.93	96.1%	1.90
P3	QWERTY	1.6, 2.0, 1.0	1.53	96.1%	1.00

These speeds are far below published dwell-based gaze typing, where novices start at several words per minute and reach about 20 after ten sessions with an adjustable dwell [6]. Our reasons: a dwell of 1.1 to 1.5 s, much longer than practised users choose; about 1.85 selections per letter on the grouped layout; first-time users copying an unfamiliar passage; a calibration error the size of a QWERTY key; and suggestions from a fixed word list rather than a language model. Grouped speed rose from the first to the third run for all three participants (Table 9), consistent with an early point on the learning curve; QWERTY varied with no clear trend.

Usability. Table 10 gives the SUS scores (0 to 100; about 68 is average). All three are below average: P1 52.5, P2 40.0 and P3 50.0 (mean 47.5). P1 and P2 rated the functions as well integrated and none found the system inconsistent, so the structure of the app held up; P3, the target user, gave the best ratings for complexity and awkwardness (Q2, Q6, Q8). The low items were about effort and confidence: no one would yet like to use it frequently or felt very confident, and P2 and P3 felt they would need technical support. The feedback points to pointer jitter and the QWERTY keyboard as causes. Inexperience is a third: all three were using gaze pointing for the first time, and P1 said that coordinating head and gaze “needs getting used to”. Part of the low scores may therefore reflect an early point on the learning curve, which the rising grouped speeds and P3’s remark that the system became easier with use both support; repeated sessions would show how much.

Table 10. System Usability Scale answers, from 1 (strongly disagree) to 5 (strongly agree). Odd items are positive statements (Q1 use frequently, Q3 easy, Q5 well integrated, Q7 quick to learn, Q9 confident) and even items negative (Q2 complex, Q4 need support, Q6 inconsistent, Q8 cumbersome, Q10 much to learn).

	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	SUS
P1	1	3	3	2	4	2	2	2	2	2	**52.5**
P2	2	3	2	4	4	2	2	3	2	4	**40.0**
P3	2	2	2	4	2	1	3	1	2	3	**50.0**

5.5 User feedback

P1 and P2 both found the pointer too shaky. P1 suggested more smoothing, like a mouse pointer speed setting, and P2 named shaky gaze as the main problem. Both found targets near the screen corners hard to reach, where gaze estimates are least accurate, and both struggled with QWERTY, whose keys are about the size of the calibration error. P1 added that coordinating head and gaze “needs getting used to”, and P2, who wore glasses, found deleting a message the hardest task. P3, from the target group, found horizontal rows of buttons harder to select than vertical ones, asked for a better colour scheme, and asked for a way to reset the cursor. P3 also saw clear merit and potential in the system for people with motor impairments, and said it became easier to use over time, which matches P3’s rising selection rate and grouped typing speed. For the next version this points to five changes: a smoothing setting, targets kept clear of the corners, larger QWERTY keys alongside the grouped default, a high contrast theme, and a reset pointer action.

5.6 Reliability, validity and bias

Reliability. Every participant followed the same protocol, setting and tasks, and all performance measures are computed by scripts from the event log, so the same data always give the same numbers. With one session per person, we cannot yet say how stable the measures are from day to day.

Validity. The evaluation measures performance through accuracy (accidents, typing accuracy, confirmation success) and speed (words per minute, selection rate), taken from logged behaviour rather than self-report, and how people felt through SUS and open feedback. SUS is a well-established measure of perceived usability, which gives construct validity. The tasks cover the main actions across all four risk tiers (navigating, changing a setting, typing and sending, calling, deleting), which gives content validity, and they are everyday messaging actions, which gives face validity.

Ecological validity is low: a lab room, an experimenter present, fictional contacts and scripted tasks. The findings do not yet generalise to motor-impaired users in daily life. The hardware is a laptop and a phone, so home testing is feasible in a future round.

Biases. Participants were a convenience sample, and two of the three were able-bodied people standing in for the target group. The experimenter was an author of the system and may have helped or encouraged without

meaning to. Participants knew they were being observed and that the team had built the app, so demand characteristics may have raised their effort and softened their ratings, although the SUS scores were still low. The keyboard order was fixed, grouped first, so practice and fatigue affect QWERTY more.

5.7 Limitations

The evaluation is small: three participants, one session each and no fixed baseline. The results show that the learners behave as intended, not that adaptation beats a fixed dwell. A more comprehensive evaluation needs more participants, including more people with motor impairments, who are the intended users.

Participants also had little time to learn the interface before the tasks. All were new to gaze pointing, and their SUS scores, comments and rising typing speeds suggest that part of the difficulty came from inexperience. Future participants should get a longer guided practice period, to become familiar with the interface and with coordinating head and gaze, before the unguided tasks begin.

6 Ethics and Risk Assessment

6.1 Risks and mitigation

Table 11. Risks to participants and how we reduce them.

Risk	Mitigation
Fatigue and eye strain	Breaks offered between parts; the person can pause or stop at any time; the dwell lengthens after errors instead of rushing the person
Discomfort (neck, posture)	Head free to move; phone placed at a comfortable distance; banners ask the person to move only when tracking fails
Frustration from errors	Undo on every action for 5 s; risky actions need a second signal; a hint after 30 s of being stuck; no time pressure except in the typing test
Privacy of face and voice	Camera images are shown live only and never stored; speech is recognised on the phone and only the word “go” is sent; nothing goes to a cloud service
Identification from data	Data is saved under a participant code (P1 to P3); names are kept apart from the data; shared evaluation data uses codes only

An experimenter supervised every session. The logs hold gaze coordinates, head position and angles, and interaction events; they hold no images, audio or names. The onboarding screen asks for an age range and a condition category as selection practice. Participants could pick any option; the answers are written to the event log like any other selection, are not analysed, and are removed from the evaluation data before it is shared.

6.2 Statement of voluntary participation

Participation is voluntary. Each participant is told what the session involves and what is recorded, gives consent before starting, and may stop at any time without giving a reason. We collect no names, contact details, images or audio. The only health-related item is the optional, self-chosen condition category from onboarding, which is not analysed and is removed before the data is shared. The data is stored on the team’s computer under participant codes and used only for this course project.

7 Conclusion

Communic-aid is a working high-fidelity prototype in which one consumer phone provides both pointing and confirmation for hands-free messaging, and the interface learns from ordinary use, without asking, how long to wait and which signal to ask for before acting. Tests with three people, one from the target group who saw real potential in it and found it easier with use, show rare, cheap, undoable accidents, no accidents on risky actions, a dwell that shortens within one session, and a router that changes signals per person. Usability scores were below average (SUS 47.5 on average), and both the heuristic evaluation and the participants point to the same fixes: a steadier, resettable pointer, larger and better placed targets, and a clearer colour scheme. The screen layout draws on the gaze-controlled Hindi keyboard of Meena et al. [7], and our grouped keyboard follows its two-step idea of a group, then a letter. Since our groups are alphabetical, a next version could order letters and groups by frequency and selection time, as their best layout did.

References

[1] John Brooke. 1996. SUS: A “Quick and Dirty” Usability Scale. In *Usability Evaluation in Industry*. Taylor & Francis, 189-194.

[2] David J. Chalmers. 2019. Extended Cognition and Extended Consciousness. In *Andy Clark and His Critics*, Matteo Colombo, Elizabeth Irvine and Mog Stapleton (Eds.). Oxford University Press, 9-20.

[3] Andy Clark and David J. Chalmers. 1998. The Extended Mind. *Analysis* 58, 1, 7-19.

[4] Robert J. K. Jacob. 1990. What You Look at is What You Get: Eye Movement-Based Interaction Techniques. In *Proceedings of CHI ’90*. ACM, 11-18.

[5] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. 2010. A Contextual-Bandit Approach to Personalized News Article Recommendation. In *Proceedings of WWW ’10*. ACM, 661-670.

[6] Päivi Majaranta, Ulla-Kaija Ahola, and Oleg Špakov. 2009. Fast Gaze Typing with an Adjustable Dwell Time. In *Proceedings of CHI ’09*. ACM, 357-360.

[7]Meena, Y. K., Cecotti, H., Wong-Lin, K., Dutta, A., & Prasad, G. (2018). Toward optimization of gaze-controlled human–computer interaction: Application to hindi virtual keyboard for stroke patients. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, *26*(4), 911–922. https://doi.org/10.1109/tnsre.2018.2814826

[8] Yogesh Kumar Meena and M. Salvi. 2025. Multimodal Appearance-Based Gaze-Controlled Virtual Keyboard with Synchronous-Asynchronous Interaction for Low-Resource Settings. arXiv:2508.16606.

[9] Martez E. Mott, Shane Williams, Jacob O. Wobbrock, and Meredith Ringel Morris. 2017. Improving Dwell-Based Gaze Typing with Dynamic, Cascading Dwell Times. In *Proceedings of CHI ’17*. ACM, 2558-2570.

[10] Omar Namnakani, Yasmeen Abdrabou, Jonathan Grizou, and Mohamed Khamis. 2025. GazeBlend: Exploring Paired Gaze-Based Input Techniques for Navigation and Selection Tasks on Mobile Devices. arXiv:2512.15491.

[11] Jakob Nielsen. 1994. Enhancing the Explanatory Power of Usability Heuristics. In *Proceedings of CHI ’94*. ACM, 152-158.

[12] R. William Soukoreff and I. Scott MacKenzie. 2003. Metrics for Text Entry Research. In *Proceedings of CHI ’03*. ACM, 113-120.

[13] William R. Thompson. 1933. On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples. *Biometrika* 25, 3/4, 285-294.

[14] Jacob O. Wobbrock, Shaun K. Kane, Krzysztof Z. Gajos, Susumu Harada, and Jon Froehlich. 2011. Ability-Based Design: Concept, Principles and Examples. *ACM Transactions on Accessible Computing* 3, 3, Article 9, 27 pages.

[15] Xiaoyi Zhang, Harish Kulkarni, and Meredith Ringel Morris. 2017. Smartphone-Based Gaze Gesture Communication for People with Motor Disabilities. In *Proceedings of CHI ’17*. ACM, 2878-2889.